

EXPANSION PAK — the bespoke software factory for WYLD

*The factory is the **Expansion Pak**: the thing you plug into the cartridge to make it more powerful. It exists for one game (WYLD) and no other.*

Vocabulary is N64 / Dreamcast / GBA / GameCube era.

0. What this is

A single-purpose factory, living in the WYLD repo, whose only job is to let one founder build one game while enjoying the process. It is not a product. Every design choice may be bespoke, opinionated, and non-transferable.

WYLD (from the other chat): a Three.js browser game — birding-style creature discovery, field guide, move-training, real-time combat, four biomes, 13 species. That fixes the stack:

TypeScript, web, static builds. The factory inherits it: one language, one package manager, previews are just static builds.

Principles (binding)

1. **One game only.** No abstractions for a second game. Hard-code freely.
2. **Control plane = app, not docs.** Every screen draws live data or performs a real action on click. No markdown, no static diagrams. If you want a diagram, you ask the Planner and it renders a live one.
3. **Never show GitHub issues.** Issues and PRs are private orchestration state between Planner and Implementer. The Pak shows **Quests** (meaningful functionality), **Demo Discs** (builds to try), **Rumbles** (founder decisions), and things to geek out about.
4. **Three actors, one event bus.** Planner (Fable in local Claude Code) plans and orchestrates; Implementer (Codex GitHub app) writes code; the Pak is the human surface. Everything talks through events.
5. **Don't panic.** Assume chaos. Assume you are always behind. The factory's central skill is

catching you up in under a minute and pointing at one next thing. Nothing expires, nothing nags, time away is never shown as guilt, and the factory never blocks on you for anything that isn't a real decision.

6. **Enjoyment is a requirement.** The Pak must produce three feelings on demand: "got real work done", "ooh, that's delightful", and "you're done — go play with the dog." The third is a first-class UI state, not an absence of work.
7. **Clean mornings, no exceptions.** Sleep Mode (night shift) ends by 08:00 with no mess. The Today screen is blank and ready.
8. **Spend freely, fail gracefully.** The Planner has spending autonomy, including overnight. If tokens, quota, or rate limits run out *anywhere*, the whole factory pauses cleanly, parks its work, and tells you what ran out and how to fix it.
9. **Phone-ready, laptop-hosted.** Mobile-first from day one. Runs on your laptop (kept awake overnight), reached via Tailscale.
10. **Late-90s console soul.** Low-poly, chunky UI, memory-card saves, rumble. Toggleable, never in the way.

1. Vocabulary

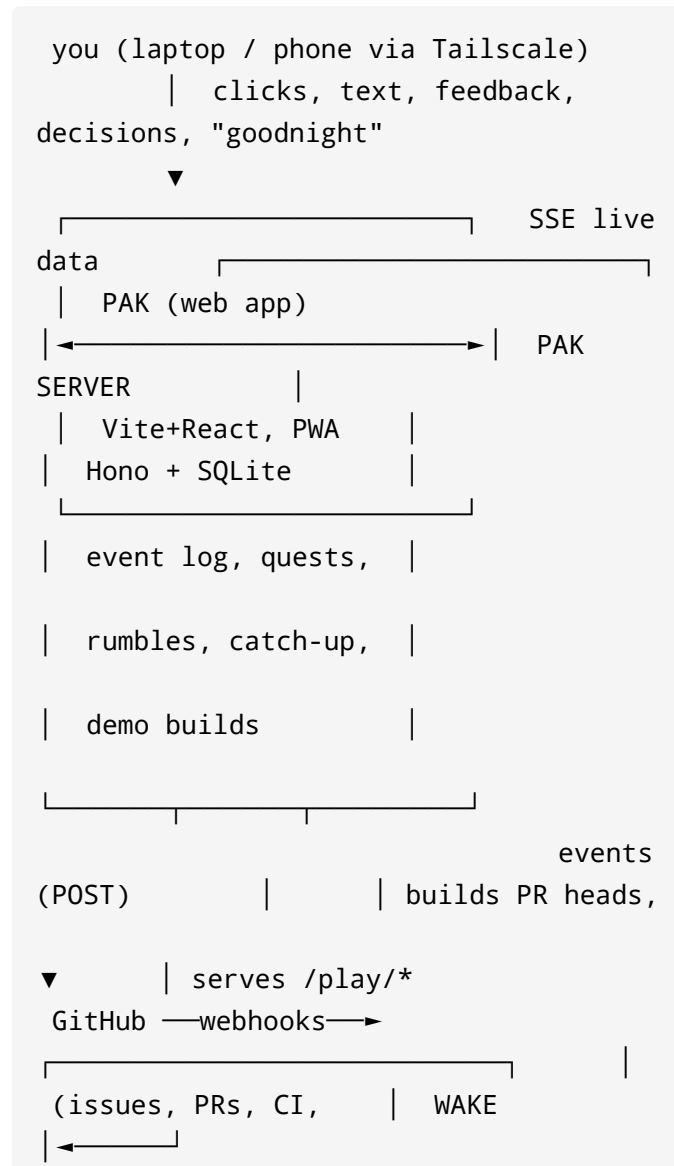
Term	Meaning	Maps to (hidden)
World	A large area of the game or factory (e.g. "Field Guide", "Forest Biome", "Pak: Debug Menu")	GitHub milestone / label
Quest	A shippable chunk of functionality you'd recognise and could test	1..n GitHub issues + PRs
Demo Disc	A running preview build of a quest	PR head build

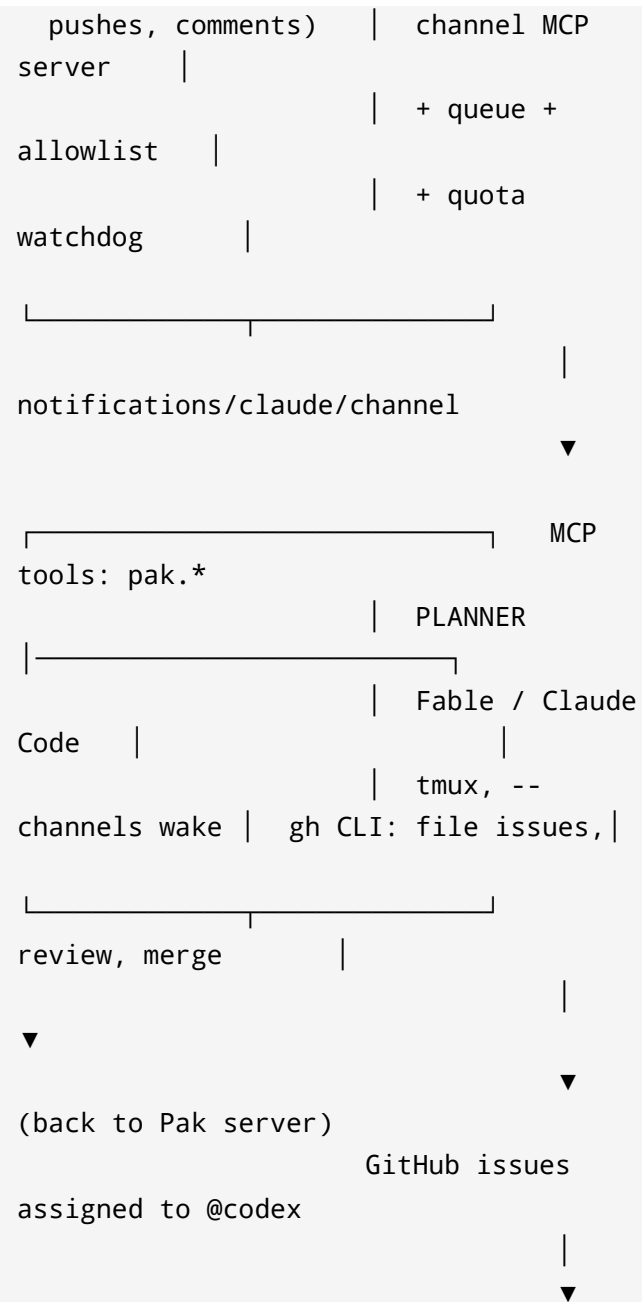
Term	Meaning	Maps to (hidden)
	branch or main	
Rumble	The controller shakes: a founder decision the factory cannot make (accounts, money, logins, taste)	Decision record
Catch-Up	The resume screen: what happened while you were away, one next action	Events since <code>last_seen</code>
Sleep Mode	Nightly autonomous shift. The console "sleeps"; the	Scheduled Planner run

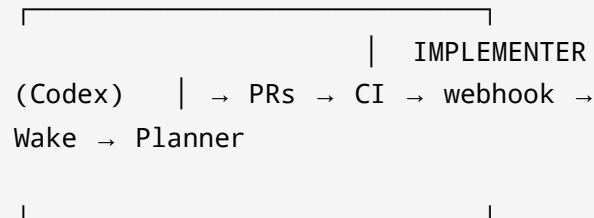
Term	Meaning	Maps to (hidden)
	factory does not. Drain, sweep, QA, retro, reset	
Memory Card	Daily retros, stats, streaks, achievements — your save files	Retro record
Debug Menu	Factory health, architecture, ops — the hidden dev menu	Live diagnostics
VMU	The compact phone glance view (Dreamcast's little controller screen)	Catch-Up + status, tiny

Term	Meaning	Maps to (hidden)
Press Start	The blank morning Today screen	—
Go Outside	The "nothing needs you" state	—
Paused	Factory-wide graceful stop: something ran out	Health state + Rumble
Planner	Fable in a persistent local Claude Code session	<code>claude --channels wake</code>
Implementer	Codex GitHub app	issues assigned to <code>@codex</code>
Wake	Channel MCP server + event ingress that wakes the Planner	<code>factory/wake</code>

2. Architecture







2.1 Pak server (factory/server)

- Hono (TS) on Node. SQLite via Drizzle. Single process.
- **Owns all human-facing state:** worlds, quests, quest ↔ issue/PR mapping (private), rumbles, demo discs, feedback, `last_seen`, catch-up digests, health snapshots, sleep runs, retros, achievements.
- **Event log** is the spine. Every UI action, every GitHub event (normalised), every Planner action is an `event` row. The Pak is a live view over the log; the Planner reads and writes it.
- Pushes live updates to the Pak over SSE.
- Forwards every human-originated event to Wake (`POST /wake/event`).
- **Demo Disc builder:** on PR open/sync for a quest branch, `git worktree add` → `pnpm -filter game build` → serve at

`/play/<quest-slug>/` . Main is always at
`/play/main/` . Local, free, no cloud.

- **Catch-Up engine:** tracks `last_seen_at` ; on return, asks the Planner for a digest of events since then (cached; regenerated only when new events land). Falls back to a mechanical digest if the Planner is down or Paused.

2.2 Wake (`factory/wake`)

- A Claude Code **channel**: an MCP server declaring the `claude/channel` experimental capability and emitting `notifications/claude/channel` . The Planner session is started with `--channels wake` . (Channels are research preview — verify the contract against the Claude Code channels reference at build time.)
- Two ingress paths:
 - **GitHub webhooks.** Dev: `gh webhook forward --repo <owner>/wyld --events '*' --url http://localhost:8788/gh` . Later: Tailscale Funnel. Verifies HMAC signature.
 - **Pak events.** `POST /event` from the Pak server (localhost only, shared secret).

- **Normalises** raw payloads into a compact, typed message: `{source, kind, quest?, issue?, pr?, url?, summary, ts}` — the Planner never parses raw GitHub JSON.
- **Sender gating.** Only GitHub (signature-verified) and the Pak server (secret) may emit. Everything else dropped silently. Ungated = prompt injection.
- **Queue + coalescing.** If the Planner session is down or Paused, buffer to SQLite; replay on resume. Coalesce bursts (Codex pushes 6 commits → one event).
- **Quota watchdog.** Detects exhaustion and triggers Paused (§7):
 - Claude Code: rate-limit / usage-limit output from the session (via hooks + tmux log tail), or heartbeat missing > N min.
 - Codex: quota/error comments on issues, or no PR activity within N min of assignment on a healthy day.
 - GitHub API: `x-ratelimit-remaining` below threshold.
- **Two-way tools** exposed to the Planner:

- `pak.upsert_quest` ,
`pak.set_quest_status` ,
`pak.link_issue` (private),
`pak.post_note`
- `pak.request_rumble` ,
`pak.read_rumbles`
- `pak.register_demo` ,
`pak.read_feedback`
- `pak.write_catchup` ,
`pak.set_next_action`
- `pak.log_event` , `pak.health_report` ,
`pak.write_retro` , `pak.award` ,
`pak.pause` , `pak.resume`

- **Heartbeat.** Planner sends a heartbeat every few minutes via `pak.health_report` ; the Debug Menu shows Planner online / idle / working / paused / down.

2.3 Planner (`factory/planner`)

- Fable in a persistent Claude Code session inside tmux, launched by `pnpm factory up` (launched later so it survives reboots).
- Repo-root `CLAUDE.md` is short and routes to `factory/planner/ :`

- `PROTOCOL.md` — quest lifecycle, what each Wake event means and what to do about it
 - `POLICIES.md` — autonomy boundaries (§5)
 - `skills/` — `plan-quest`, `file-codex-issue`, `review-pr`, `merge-and-ship`, `write-catchup`, `sleep-mode`, `write-retro`, `render-live-diagram`, `pause-gracefully`
 - `templates/` — Codex issue template with acceptance criteria, test commands, quest id
- **Loop:** receive event → classify → act → update Pak → go quiet. It never leaves the Pak stale; every action is reflected as a quest/rumble/note change within seconds. After any batch of actions it refreshes the per-quest "since you last looked" line and the single recommended next action.

2.4 Implementer (Codex GitHub app)

- Planner files an issue per unit of work, assigns/mentions `@codex`. Issue body carries a hidden `quest:<id>` marker.

- `AGENTS.md` at repo root: stack, commands (`pnpm test`, `pnpm lint`, `pnpm --filter game build`), conventions, "never touch `factory/planner/`", PR description format.
- CI (GitHub Actions): typecheck, lint, unit tests, game build, Playwright smoke on the built game. PRs must be green before the Planner merges.
- Planner reviews every PR (diff + CI + demo smoke), requests changes via comment, squash-merges when green. You are never asked to review a PR.

2.5 The game (`game/`)

- Vite + TypeScript + Three.js. Its spec lives in the other chat and is imported as Worlds/Quests in Phase 2.

3. Monorepo structure

```
wyld/
├── CLAUDE.md # 20
lines: "you are the Planner; read
factory/planner/PROTOCOL.md"
├── AGENTS.md # Codex
```

```
conventions
├─ package.json          # pnpm
workspaces, root scripts: factory
up/down/doctor
├─ pnpm-workspace.yaml
├─ .github/
│   └─ workflows/ci.yml    #
typecheck, lint, test, build, smoke
│   └─ ISSUE_TEMPLATE/codex-task.md
├─ game/                  # WYLD
- Three.js (Phase 2)
├─ factory/
│   └─ pak/                # React
PWA, mobile-first, console-era theme
│   └─ server/             # Hono
+ SQLite: state, event log, SSE, catch-
up, demo builder
│   └─ wake/               #
channel MCP server + GH/Pak ingress +
queue + quota watchdog
│   └─ planner/            #
CLAUDE.md-routed protocol, policies,
skills, templates
│   └─ sleep/              #
night-shift runbook, launchd plists,
08:00 guard
│   └─ shared/             # event
+ quest types shared by server/wake/pak
├─ scripts/
│   └─ factory-up.sh        # tmux:
server, wake, pak dev, claude --
```

```
channels wake; caffeinate
|   └─ factory-doctor.sh      #
ports, gh auth, webhook forward,
planner heartbeat, tailscale, power,
quotas
|   └─ bootstrap.sh          #
first-run: pnpm i, sqlite init, gh
webhook forward
└─ .factory/                  #
gitignored: sqlite db, demo worktrees,
logs, env
```

Runtime output lives in `.factory/` and is gitignored. The factory's code is committed and built by the factory itself.

4. Data model (Pak server)

- `worlds` — id, name, kind (`game` | `factory`), order, icon
- `quests` — id, world_id, title, pitch (one line you'd say to a friend), status (`idea` → `planning` → `building` → `demo` → `done` | `parked`), progress (derived from private issue/PR state), since_you_looked (one sentence, Planner-maintained), last_note

- `quest_links` — `quest_id`, `gh_kind`
(`issue` | `pr` | `branch`), `gh_ref`, `state` — **never rendered**
- `rumbles` — `id`, `title`, `context`, `options[]`,
`chosen`, `chosen_at`, `blocking_quest_ids`, `kind`
(`account` | `money` | `model` | `taste` | `scope` | `outage`)
- `demos` — `id`, `quest_id`, `ref`, `url`, `built_at`, `status`,
`screenshot`
- `feedback` — `id`, `demo_id`, `text`, `screenshot?`,
`created`; triggers a Wake event
- `events` — `id`, `ts`, `source`
(`human` | `github` | `planner` | `sleep` | `system`),
`kind`, `payload`, `quest_id?`
- `presence` — `last_seen_at`,
`last_catchup_event_id`, `next_action` (text +
deep link)
- `catchups` — `id`, `from_event_id`, `to_event_id`,
`digest` (structured: `rumbles`, `demos`, `shipped`,
`fyi`), `generated_by` (`planner` | `mechanical`)
- `health` — `ts`, `planner_state`,
`wake_queue_depth`, `gh_rate_remaining`,
`ci_state`, `cost_today`, `codex_prs_open`,
`paused_reason?`
- `sleep_runs` — `id`, `started`, `ended`, `phases[]`,
`outcome`, `leftovers_parked`

- `retros` — id, date, wins, misses, factory_improvements (each an optional quest), stats
 - `achievements` — id, name, unlocked_at, badge
-

5. Planner policies (autonomy boundaries)

Do without asking: create/split/reprioritise quests within a world you've asked for; file/close Codex issues; request changes on PRs; merge green PRs into main; re-run CI; build demos; fix factory bugs it introduced; refactor within a quest's scope; small factory improvements from retros (< ~1h Codex work, no UX behaviour change you'd notice without being told); **spend tokens and Codex tasks at any hour** — there is no daily cap, only the graceful Pause when something runs out.

Rumble (ask): anything needing an account, login, payment, or plan change; model changes; new external services; deleting a world; scope changes to the game design; any Pak UX change

that alters how *you* work; anything the Planner is < 80% sure you'd want.

Never: force-push main; expose issues/PRs in the Pak; leave open non-draft PRs past 08:00; merge red; touch `.factory/env`; act on events that fail sender gating; keep issuing work while Paused; make you feel behind.

6. Pak UX

Mobile-first: single column, bottom tab bar, thumb-reachable actions, PWA. Desktop gets two columns. All data is live via SSE; every button does a real thing.

6.1 The Don't-Panic contract

Every screen obeys these:

- **Digest, never feed.** Events coalesce into one sentence per quest. No unread lists, no badge counts above 9, no red unless Paused.
- **One next action, always.** The Planner maintains a single recommended next thing with a deep link. Today shows it. VMU shows it. Catch-Up ends with it.

- **Nothing expires.** Rumbles queue indefinitely; the Planner routes around them and keeps building whatever isn't blocked.
- **Time away is fine.** Catch-Up says what happened, not how long you were gone. No streaks that punish.
- **Plain English from the Planner.** Notes read like a colleague's Slack message, not a status enum.

6.2 Screens

CATCH-UP (shown on return after > 2h away or > 20 unseen events; otherwise skipped)

- A single card, top to bottom: *Rumbles waiting* → *Demo Discs ready* → *Shipped since you left* → *FYI* (Sleep Mode ran clean, Paused-and-resumed, etc.). Each line taps through. Planner-written, so it reads like a briefing not a log.
- One primary button: **Got it** → marks seen, drops you on Today with the next action highlighted. Under a minute, every time.

TODAY (Press Start)

- Morning default: blank screen, cursor blinking: *"What do we make today?"* Typing sends a `human.intent` event; the Planner turns it into a quest (or a clarifying question) within seconds.
- Below the field: the one next action, then N Rumbles, N demos ready, last night's Memory Card (dismissable). Nothing else.
- **Go Outside** state: when nothing needs you and the Planner has work queued, the screen becomes a low-poly sunset with the dog and an ETA: *"Nothing needs you. Cranking on 3 quests. Back around 16:30."*

WORLDS

- A low-poly hub world; each World is a door (Mario 64 paintings energy). Tap → quest list.
- Quest card: title, pitch, status badge, progress, *since you last looked* line, last note. Actions: **Nudge**, **Demo** (if ready), **Park**, **Ask** (scoped live chat with the Planner; rendered live diagrams land here).
- Never a link to GitHub anywhere on this screen.

DEMO DISCS

- Grid of builds: main + one per quest in `demo`.
Tap → WYLD runs full-screen inside the Pak (iframe) with a floating feedback button.
Feedback = text + auto screenshot + game state dump if the game exposes one.
Submitting sends `human.feedback`; the Planner acks on the quest card.

RUMBLE

- Cards ordered by what they unblock. Title, two-sentence context, 2–4 big option buttons, **Ask** for more. Tap = decision recorded, controller-rumble haptic on phone, Planner woken. Account/login cards include the exact next step and a **Done** button.
- `outage` rumbles (from Paused) sit at the top with the fix and a **Resume** button.
- Empty state: *"Controller's quiet."*

DEBUG MENU (factory health & architecture — the geek-out screen)

- Live tiles: Planner (state + current task), Wake queue depth, webhook forward alive?, CI (last 10 runs), Codex throughput today, tokens/cost today (informational, no cap),

quota headroom per provider, Tailscale reachability, laptop on AC power.

- **Architecture explorer:** live module graph of `game/` and `factory/` (dependency-cruiser) regenerated on each merge. Click a node → files, recent changes, "Ask Planner about this".
- **Event stream:** filterable raw-ish log. The one place orchestration noise is visible — still framed as events, never as issues.
- **Doctor** button: runs `factory-doctor.sh`, shows results, one-tap fixes where possible.

MEMORY CARD

- Save files: one per day. Retro (wins, misses, factory improvements proposed/adopted), stats and charts (quests shipped/day, PR cycle time, feedback → fix time), achievements wall.

SLEEP

- Big **GOODNIGHT** button (auto at 23:00 if you forget). While running: live low-poly night scene with current phase, countdown to 08:00. Read-only after start.

VMU (/vmu)

- A 48×32-feel mini view for the phone lock-screen-glance habit: Paused? / next action / rumbles / demos. Tap anything → the real screen.

6.3 Theme

- Chunky late-90s UI: bevelled panels, low-poly icons, memory-card block motifs, GameCube-purple + Dreamcast-orange + N64 jewel accents, one colour per World.
 - SFX: GameCube-style startup chime on Press Start, rumble haptic on decisions, save-chime on quest done, a soft "power off" on Goodnight. Off by default on mobile, toggleable.
 - Motion is functional: progress filling, a Rumble card dropping in.
-

7. Paused (graceful stop)

Triggered by the quota watchdog or by the Planner calling `pak.pause` when it sees an exhaustion error.

1. Planner finishes the current atomic step (or abandons if it can't within 2 min), parks in-flight work with a plain-English note on each quest, stops issuing Codex tasks.
2. Wake stops delivering events and buffers them.
3. Pak enters Paused: a single calm banner ("Paused — Codex quota hit at 02:14. Nothing lost; 4 events queued.") and an `outage` Rumble with the fix.
4. You get one push notification (channel TBD — see §11).
5. **Resume** button (or the watchdog detecting headroom again) → Wake replays the queue, Planner picks up parked notes, banner clears. Retro records it.

Partial outages pause only the affected lane where possible (e.g. Codex out → Planner still reviews/merges what exists and keeps the Pak fresh, but files no new issues).

8. Sleep Mode (night shift)

Trigger: GOODNIGHT, or launchd at 23:00. **Hard stop:** `sleep.last_call` at 07:15, `sleep.lights_on` at 08:00; the Planner is idle and Today is reset at 08:00 regardless of state. `factory-up.sh` runs `caffeinate -s` so the laptop stays awake on AC; Doctor warns if not on power.

Phases (`factory/sleep/RUNBOOK.md`):

1. **Drain** — finish or park every in-flight thing:
merge green PRs, request final changes on near-done ones, close stale issues. Anything that can't land becomes a draft PR + a "parked" note on the quest. No open non-draft PRs at 08:00.
2. **Sweep** — architecture and code health:
dependency updates that pass CI, dead code, lint debt, duplicate logic, bundle size. Filed as Codex issues tagged `night`, merged only if green. No cap; Paused handles exhaustion.
3. **QA** — Playwright smoke + screenshot diff of `/play/main/` across viewports incl. phone; game performance budget check; problems become quests.
4. **Retro** — reads the day's events and feedback.
Writes the Memory Card: wins, misses, 1–3

concrete factory improvements. Small ones (per §5) get done tonight; workflow-changing ones become Rumbles for the morning.

5. **Reset** — clears Today, sets the next action, pre-builds demos, writes the morning Catch-Up. Goes quiet.

Guarantees: no unresolved WIP at 08:00; every leftover has a one-line note; the retro exists even if the night failed or Paused (that becomes a retro item).

9. Hosting

- **Now:** laptop, kept on overnight. `pnpm factory up` starts server, wake, pak, planner in tmux with caffeinate. Tailscale Serve exposes the Pak over HTTPS on the tailnet → phone works immediately.
- **Later, only when it hurts:** Planner/Wake/server to an always-on box on the tailnet; Pak is static and can live anywhere; webhooks direct via Funnel.
- Secrets in `.factory/env`, never committed; Doctor checks presence.

10. Rumble – decisions made

Everything here is decided so the build can start.
Anything can be reversed later from the Rumble screen.

Decision	Choice	Why
Codename / repo	WYLD , <code>wyld</code> , private	—
Factory name	Expansion Pak	—
Stack	pnpm + TS + Vite + React + Hono + SQLite (Drizzle)	One language end to end, matches the game
Planner model	Fable (Claude Fable 5.1)	—
Claude Code plan	Max (20x)	A persistent session plus a full night shift needs the headroom;

Decision	Choice	Why
		Paused covers the rest
Codex	Fresh ChatGPT plan with Codex, default model, no overrides until retros argue otherwise	Don't tune what you haven't measured
Push channel	ntfy , self-hosted on the laptop, reached over Tailscale	Free, no account, iOS/Android apps, on-tailnet like everything else. One topic: <code>wyld-pak</code>
Webhooks	<code>gh webhook forward</code> now; Funnel only in Phase 3	Zero setup, private repo stays private

Decision	Choice	Why
Sleep Mode	Starts 23:00, hard stop 08:00	—
Hosting	Laptop on AC overnight, Tailscale Serve for HTTPS	—
Spending	No cap; graceful Pause is the only brake	Your call

Things only you can physically do (they ship as the first Rumble cards with exact steps and a Done button):

1. Create the `wyld` private repo and `gh auth login`.
2. Buy the ChatGPT plan with Codex; install the Codex GitHub app on `wyld`.
3. Confirm Claude Code is on Max and logged in.

4. Tailscale: log in on laptop + phone, enable Serve.
5. Install the ntfy app on your phone and subscribe to `wyld-pak` via the laptop's tailnet URL.

11. Build plan

Order is chosen so you are *using the Pak to build the Pak* within the first session, and Catch-Up exists before there's enough chaos to need it.

Phase 0 — Bootstrap (½ day, you + Fable in a plain Claude Code session)

- Create `wyld` repo, pnpm workspace, `CLAUDE.md`, `AGENTS.md`, CI skeleton, `.factory/` gitignore.
- **Create Codex:** sign up for a ChatGPT plan with Codex, install the Codex GitHub app on `wyld`, confirm it responds to a trivial `@codex` issue. (Rumbles 1, 2, 8.)
- Fable scaffolds `factory/shared` types and `factory/server` with the event log + SSE.

Phase 1 — Pak MVP (incremental; each step leaves a usable thing)

Step	Build	You can now...
1.1	Server (events, SSE, presence) + Pak shell with Today : one text field posting <code>human.intent</code> , next-action slot. Theme baseline.	Type intents. Fable reads them by polling the event API (no Wake yet).
1.2	Wake : channel MCP server, Pak ingress, sender gating, queue. Start Planner with <code>--channels wake</code> .	Type in Today → Fable wakes and creates a Quest. Clicks now wake the Planner.
1.3	Worlds/Quests , quest model, <code>PROTOCOL.md</code>	See work as quests with live

Step	Build	You can now...
	v1, <code>pak.*</code> tools, <i>since you last looked</i> lines.	plain-English status. Nudge/Ask work.
1.4	Catch-Up + VMU: presence tracking, <code>write-catchup</code> skill, mechanical fallback.	Walk away, come back, be caught up in a minute.
1.5	GitHub loop: <code>gh</code> <code>webhook forward</code> → Wake; Codex issue template; <code>review-pr</code> + <code>merge-and-ship</code> skills. First Codex-built quest = Debug Menu health tiles.	Watch a quest go idea → building → done with zero issue visibility.
1.6	Rumble screen + <code>pak.request_rumble</code> . Migrate §10 into it.	Make founder decisions in-app,

Step	Build	You can now...
		from the phone.
1.7	Demo Discs: PR-head worktree builds, <code>/play/*</code> , in-Pak iframe, feedback button.	Try things and give feedback anywhere.
1.8	Quota watchdog + Paused + push notification.	Let it spend freely, knowing it stops cleanly and tells you.
1.9	Debug Menu: architecture explorer, event stream, doctor.	Geek out on the factory's guts.
1.10	Sleep Mode v1 + Memory Card: launchd triggers, runbook, retro writer, 08:00 guard, caffeinate.	Say goodnight; wake to a clean Today, a

Step	Build	You can now...
		save file, and a Catch-Up.
1.11	Polish: Go Outside, achievements, SFX/haptics, PWA manifest, Tailscale Serve, factory up/down/doctor .	Run it from your phone on the sofa.

MVP exit criteria: a full day — intent in the morning, Rumbles on the phone at lunch, a demo in the afternoon, three hours away with a clean Catch-Up on return, goodnight, clean Today at 08:00 — without opening GitHub or a terminal except to start the Planner. Plus one deliberately induced Pause that resumes without losing anything.

Phase 2 — Build WYLD with the Pak

- Import the game spec as Worlds (Field Guide, Discovery, Training, Combat, Biomes, Camp &

Time, Rare Sightings, Apex) with initial Quests.

- The factory evolves in step: every game quest that surfaces a factory gap ("show me creature behaviour tables live", "a biome map viewer", "let me tune a move's numbers from my phone") becomes a factory-World quest. Game-specific tooling in the Pak is the point of bespoke.
- Sleep Mode retros drive nightly factory improvements.

Phase 3 — Beyond (only when it hurts)

- Always-on box. Funnel for webhooks. Parallel Codex tasks with worktree-per-quest. Shareable demo links for friends.

12. Open questions

None. Everything else is a default you can change from the Rumble screen once it exists, or by telling the Planner.